PPE Gate: Software Guide (using the Antigravity IDE)

What this guide covers: everything on the software side: installing the tools, setting up Antigravity, getting the AI model, linking the laptop to the Arduino, the Telegram alerts, the main program, the report dashboard, and testing. It is written for a beginner and uses Windows commands (Mac and Linux differences are noted).

How the work is split: the Hardware Build Guide and the Wire-by-Wire Connection Guide cover the Arduino, the wiring and the gate. This guide starts where they end. **You can start most of the software before the hardware is ready**, because the main program has a simulation mode (Phase F).

Estimated time: about 10-14 hours of software work, plus waiting time for installs and training.

---

1. What the software does

```
Webcam --> Laptop (Python + YOLO) <--USB--> Arduino (outputs, distance sensor)
                |
                +--> logs/events.csv + snapshots/ --> Streamlit dashboard
                +--> Internet --> Telegram bot --> your phone
```

Piece	File	Job
Camera test	`test_camera.py`	Check the webcam works
Arduino test	`test_arduino.py`	Send commands to the Arduino
Distance test	`test_distance.py`	Read the ultrasonic sensor
Telegram test	`test_telegram.py`	Send a photo alert
Main program	`main.py`	Detect helmet and vest, decide, control the gate, log, alert
Dashboard	`dashboard.py`	Show the compliance report
Secrets	`secrets_config.py`	Your Telegram token and chat ID (never share)
AI model	`best.pt`	The trained YOLO model

The decision flow (state machine): IDLE (waiting for someone to step close) > CHECKING (the AI looks for a helmet and vest for about 3 seconds) > RESULT (green gate opens, or red denies with a photo and a Telegram alert) > WAIT_LEAVE (waits for the person to walk away) > back to IDLE.

---

2. What to install

Tool	Why	Where from
Antigravity IDE	Your code editor, with an AI agent	antigravity.google (Download page)
Python 3.10 or 3.11 (64-bit)	Runs the program	python.org. On the first installer screen tick **"Add Python to PATH"**.
Arduino IDE	Uploads the sketch to the Arduino. **Keep it even though you use Antigravity**, because it handles the board, the port and the libraries easily.	arduino.cc
CH340 driver (only if Windows does not detect a clone Arduino)	Lets the laptop see the board	Search "CH340 driver"
Telegram on your phone	Receives alerts	Play Store or App Store
Google account	Sign in to Antigravity and Google Colab	
Git (optional)	To clone the GitHub repo. You can also download it as a ZIP.	git-scm.com

---

3. Setting up Antigravity

Antigravity is a code editor that looks like VS Code and has an AI agent built in. Its screens and menu names can change between versions, so if something below looks different on your screen, use the closest matching option.

Step 1: install and sign in.

1. Download the installer from antigravity.google and install it.
2. On the first launch, a setup wizard asks a few questions. Pick a theme, and sign in with your Google account when asked. You can import VS Code settings or skip that.

Step 2: choose how much freedom the agent has. The wizard asks how autonomous the agent should be. For a beginner, choose the **safer options**:

- Prefer the setting where the agent **asks you before running terminal commands**.
- Prefer the setting where the agent **shows you a plan or lets you review** before making large changes.

Avoid "always proceed" while you are learning, because the agent could install packages or change files without you noticing.

Step 3: open your project.

1. Create an empty folder on your computer called `ppe-gate` (for example `C:\Users\you\ppe-gate`).
2. In Antigravity choose **Open Folder** and pick it.
3. The editor view looks like a normal code editor: files on the left, code in the middle, an agent panel on the side. The **Agent Manager** is a separate view for bigger agent tasks. For this project the editor view with the agent panel is enough.

Step 4: open a terminal inside it. Use the menu **Terminal > New Terminal**. It opens at the bottom, already inside your folder. All the commands in this guide go there.

Step 5: Python support. If the editor offers to install the Python extension, accept. Later, press **Ctrl+Shift+P**, type **Python: Select Interpreter**, and pick the one inside `venv` (you create it in Phase A). If the extension is not available, you can still run everything from the terminal.

Step 6: give the agent project rules. Create a file called `AGENTS.md` in your folder with the text below. The agent reads it before working, so it follows your constraints.

```
# Rules for the agent (PPE Compliance Gate, student project)
- Keep the code simple, with clear comments. The student must be able to explain it in a viva.
- Language: Python 3.10/3.11. Do not use fancy frameworks.
- Never write or print secrets. The Telegram token lives only in secrets_config.py.
- Do not change the serial protocol: laptop to Arduino sends one letter
  C, G, R, H, V or I; Arduino to laptop sends D:<cm> lines. Baud rate 9600.
- Always tell me what you changed and how to test it.
- Ask before installing packages or running anything outside this folder.
```

How to work with the agent safely

- Treat the agent like a helper, not a boss. **Read what it writes** and run the small test scripts yourself.
- **Never paste your Telegram token into the agent chat.** Put it in `secrets_config.py` yourself.
- Ask it to **explain** code you do not understand. Examiners will ask, and you need to know.
- If a change breaks something, say what broke and paste the exact error text.

---

4. Project folder and packages

Your folder should end up like this:

```
ppe-gate/
  AGENTS.md
  main.py
  dashboard.py
  test_camera.py
  test_arduino.py
  test_distance.py
  test_telegram.py
  secrets_config.py      <- your token (never share)
  requirements.txt
```

```
best.pt               <- the AI model
logs/ and snapshots/  <- created automatically
```

Create `requirements.txt` with these lines:

```
ultralytics
opencv-python
pyserial
requests
streamlit
pandas
```

Phase A: Create the environment (30-60 minutes)

In the Antigravity terminal:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

On Mac or Linux use `source venv/bin/activate` instead of the second line. The install is large (several GB, because PyTorch comes with ultralytics), so use good WiFi and expect to wait. The terminal prompt should show `(venv)` at the start when the environment is active. **You must activate it every time you open a new terminal.**

Done when: `python -c "import ultralytics, cv2, serial; print('ok')"` prints `ok`.

Phase B: Test the camera (10 minutes)

Create `test_camera.py`:

```
import cv2

CAMERA_INDEX = 0   # try 1 if you get a black or missing image

cap = cv2.VideoCapture(CAMERA_INDEX)
if not cap.isOpened():
    raise SystemExit("Cannot open camera. Try CAMERA_INDEX = 1 and close Zoom or Teams.")

while True:
    ok, frame = cap.read()
    if not ok:
        break
    cv2.imshow("Camera test (press q to quit)", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Run `python test_camera.py`. **Done when:** you see yourself, and `q` closes the window.

Phase C: Get the AI model (30 minutes to half a day)

You have two options. **Option 1 is faster, and you should try it first.**

Option 1: use a pre-trained model from the GitHub project

The Construction-PPE-Detection project (github.com/Ansarimajid/Construction-PPE-Detection) ships a trained model in its `Model` folder, named `ppe.pt`. It is MIT licensed, so you may use it. Keep the license notice and cite the project in your report.

1. Open the GitHub page. Click **Code > Download ZIP** (or clone it with Git).
2. Unzip it, find `Model/ppe.pt`, and copy it into your `ppe-gate` folder. Rename it `best.pt`, or set `MODEL_PATH = "ppe.pt"` in `main.py`.
3. **Check the file size.** A real model is several megabytes. If it is only a few kilobytes, the download is a placeholder and not the model, so download it again from the GitHub page directly.
4. See which classes the model knows. Run:

```
python -c "from ultralytics import YOLO; print(YOLO('best.pt').names)"
```

The model should list classes such as Hardhat, NO-Hardhat, Safety Vest, NO-Safety Vest and Person. `main.py` matches names in lower case. If your model uses different names (for example `safety_helmet`), add them to `HELMET_NAMES` or `VEST_NAMES` at the top of `main.py`.

Option 2: train your own in Google Colab (the guide's original route)

1. On roboflow.com open **Universe**, search "hard hat vest" or "PPE detection", and pick a dataset with a few thousand images and a license that allows use. Download it in **YOLOv8** format ("show download code").
2. On colab.research.google.com open a new notebook, set **Runtime > Change runtime type > T4 GPU**, and run:

```
!pip install -q ultralytics roboflow
```

3. Paste the Roboflow download code (it creates a `dataset` variable). Then run:

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
model.train(data=f"{dataset.location}/data.yaml", epochs=50, imgsz=640)
```

4. When it finishes (about 30-60 minutes), download `runs/detect/train/weights/best.pt` (or `train2` if you trained twice) using Colab's file panel, and put it in your folder.
5. **Keep for your report:** `results.png`, `confusion_matrix.png`, and the mAP50 score printed in the console.

Make it work on YOUR helmet and vest

A public model was trained on real construction sites. Your classroom, lighting and helmet may differ. After Phase F, if detection is weak:

1. Take 150-300 photos of your own helmet and vest, on 3-4 people, with different angles, distances and lighting, plus some with only one item and some with none.
2. In Roboflow create an Object Detection project, upload them, and draw boxes around each helmet and vest. Export as YOLOv8.
3. Retrain in Colab starting from your first model:

```
model = YOLO("best.pt")   # upload your first model to Colab
model.train(data=f"{dataset.location}/data.yaml", epochs=40, imgsz=640)
```

4. Download the new `best.pt` and replace the old one.

Done when: `best.pt` is in the folder and you know its class names.

Phase D: Link the laptop and the Arduino (needs the finished hardware)

Do this once the hardware passes its tests. First find the **COM port**: in Arduino IDE look at **Tools > Port**, or in Windows open Device Manager > Ports (COM & LPT). **Close the Arduino Serial Monitor before running any Python script**, because only one program can use the port.

`test_arduino.py` sends commands one by one:

```
import time
import serial

SERIAL_PORT = "COM3"   # change to your port

ser = serial.Serial(SERIAL_PORT, 9600, timeout=1)
time.sleep(2)   # the Arduino restarts when the port opens

for cmd, label in [("C", "checking"), ("G", "granted"), ("H", "helmet missing"),
                   ("V", "vest missing"), ("R", "both missing"), ("I", "idle")]:
    print("Sending", cmd, "-", label)
    ser.write(cmd.encode())
    time.sleep(3)

ser.close()
```

`test_distance.py` prints the sensor readings:

```
import serial
```

```
SERIAL_PORT = "COM3"   # change to your port

ser = serial.Serial(SERIAL_PORT, 9600, timeout=1)
print("Move your hand in front of the sensor. Press Ctrl+C to stop.")
while True:
    line = ser.readline().decode(errors="ignore").strip()
    if line.startswith("D:"):
        print(line[2:] + " cm")
```

Done when: the first script makes the gate go through each state in turn, and the second prints numbers that change when you move your hand.

Phase E: Telegram bot (30-45 minutes)

1. On Telegram search **@BotFather**, send `/newbot`, choose a name and a username ending in `bot`. BotFather replies with a **token**. Treat it like a password.
2. Open a chat with your new bot, press **Start**, and send any message.
3. In a browser open `https://api.telegram.org/botYOUR_TOKEN/getUpdates` (put your real token in). Find `"chat":{"id":123456789` in the text. That number is your **chat ID**. (For a group: add the bot to the group, send a message there, and use the negative number you find.)
4. Create `secrets_config.py`:

```
BOT_TOKEN = "paste-your-token-here"
CHAT_ID = "paste-your-chat-id-here"
```

5. Create `test_telegram.py`, put any picture named `sample.jpg` in the folder, and run it:

```
import requests
from secrets_config import BOT_TOKEN, CHAT_ID

url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendPhoto"
with open("sample.jpg", "rb") as f:
    r = requests.post(url, data={"chat_id": CHAT_ID, "caption": "PPE gate test alert"},
                      files={"photo": f}, timeout=10)
print(r.status_code, r.text)
```

Done when: it prints `200` and the photo arrives on your phone. A `401` means the token is wrong. "chat not found" means the chat ID is wrong or you did not press Start in the bot chat.

If the token ever leaks (for example in a screenshot or on GitHub), send `/revoke` to BotFather and use the new one.

Phase F: The main program (2-4 hours with tuning)

Create `main.py` with the code below. It does the whole job: camera, YOLO detection, the distance-triggered state machine, serial commands to the Arduino, logging, snapshots and Telegram alerts.

Simulation mode: set `SERIAL_PORT = None` to run without the Arduino. Then press **n** in the video window to pretend a person is near, and **f** to pretend they walked away. The "Arduino" commands are printed in the terminal instead. This lets you test the logic before the hardware is ready.

```
import csv
import os
import threading
import time
from datetime import datetime

import cv2
import requests
import serial
from ultralytics import YOLO

try:
    from secrets_config import BOT_TOKEN, CHAT_ID
except ImportError:
    BOT_TOKEN, CHAT_ID = None, None

# ------------------------ SETTINGS ------------------------
MODEL_PATH = "best.pt"
CAMERA_INDEX = 0
SERIAL_PORT = "COM3"       # use None to run without the Arduino (keys n and f)
BAUD_RATE = 9600
CONFIDENCE = 0.5           # lower = detects more but makes more mistakes
```

```
IMG_SIZE = 640             # use 480 if the video is laggy
FRAMES_NEEDED = 5          # consecutive frames with helmet AND vest to pass
MIN_SEEN_FRAMES = 3        # frames an item must be seen to count as present
TRIGGER_CM = 100           # wake the gate when someone is closer than this
LEAVE_CM = 120             # person has left when farther than this
HOLD_SEC = 0.6             # distance must hold this long (avoids false triggers)
CHECK_WINDOW_SEC = 3.0     # time the AI has to find helmet and vest
GRANT_OPEN_SEC = 5.0       # how long the gate stays open
DENY_SHOW_SEC = 3.0        # how long the red state stays
ALERT_COOLDOWN = 30        # seconds between Telegram alerts

HELMET_NAMES = {"hardhat", "helmet", "hard hat", "hard-hat", "safety_helmet", "safety helmet"}
VEST_NAMES = {"safety vest", "safety_vest", "safety-vest", "vest", "reflective vest"}

LOG_FILE = "logs/events.csv"
os.makedirs("logs", exist_ok=True)
os.makedirs("snapshots", exist_ok=True)

# ------------------------- ARDUINO LINK -------------------------
distance_cm = 999
ser = None
if SERIAL_PORT:
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)   # the Arduino restarts when the port opens

    def read_serial():
        global distance_cm
        while ser.is_open:
            try:
                line = ser.readline().decode(errors="ignore").strip()
                if line.startswith("D:"):
                    distance_cm = int(line[2:])
            except ValueError:
                pass
            except Exception:
                break

    threading.Thread(target=read_serial, daemon=True).start()

last_cmd = None


def send(cmd):
    """Send a one-letter command, only when it changes."""
    global last_cmd
    if cmd == last_cmd:
        return
    last_cmd = cmd
    if ser:
        ser.write(cmd.encode())
    print("-> Arduino:", cmd)


# ------------------------- LOGGING AND ALERTS -------------------------
def log_event(result, missing, snapshot=""):
    try:
        new_file = not os.path.exists(LOG_FILE)
        with open(LOG_FILE, "a", newline="") as f:
            writer = csv.writer(f)
            if new_file:
                writer.writerow(["timestamp", "result", "missing", "snapshot"])
            writer.writerow([datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                             result, "+".join(missing), snapshot])
    except PermissionError:
        print("Could not write the log. Close events.csv if it is open in Excel.")


def save_snapshot(frame):
    path = "snapshots/" + datetime.now().strftime("%Y%m%d_%H%M%S") + ".jpg"
    cv2.imwrite(path, frame)
    return path


last_alert = 0


def send_telegram_alert(missing, photo_path):
    global last_alert
    if not BOT_TOKEN or not CHAT_ID:
        return
    if time.time() - last_alert < ALERT_COOLDOWN:
        return
    last_alert = time.time()

    def _send():
```

```
        try:
            caption = ("PPE VIOLATION\nMissing: " + ", ".join(missing) +
                       "\nTime: " + time.strftime("%d-%b-%Y %H:%M:%S"))
            with open(photo_path, "rb") as f:
                requests.post("https://api.telegram.org/bot" + BOT_TOKEN + "/sendPhoto",
                              data={"chat_id": CHAT_ID, "caption": caption},
                              files={"photo": f}, timeout=10)
        except Exception as e:
            print("Telegram alert failed:", e)

    threading.Thread(target=_send, daemon=True).start()


# ------------------------- AI DETECTION -------------------------
model = YOLO(MODEL_PATH)
print("Model classes:", model.names)


def detect(frame):
    """Return (helmet_seen, vest_seen, annotated_frame)."""
    result = model(frame, imgsz=IMG_SIZE, conf=CONFIDENCE, verbose=False)[0]
    helmet = vest = False
    for box in result.boxes:
        name = model.names[int(box.cls[0])].lower()
        if name in HELMET_NAMES:
            helmet = True
        elif name in VEST_NAMES:
            vest = True
    return helmet, vest, result.plot()


# ------------------------- MAIN LOOP -------------------------
cap = cv2.VideoCapture(CAMERA_INDEX)
if not cap.isOpened():
    raise SystemExit("Cannot open the camera. Try CAMERA_INDEX = 1.")

GREY, ORANGE, GREEN, RED = (90, 90, 90), (0, 140, 255), (0, 150, 0), (0, 0, 200)
state = "IDLE"
state_start = time.time()
near_since = None
far_since = None
streak = 0
helmet_frames = 0
vest_frames = 0
send("I")

while True:
    ok, frame = cap.read()
    if not ok:
        break
    now = time.time()
    shown = frame

    key = cv2.waitKey(1) & 0xFF
    if key == ord("q"):
        break
    if SERIAL_PORT is None:
        if key == ord("n"):
            distance_cm = 50
        if key == ord("f"):
            distance_cm = 200

    is_near = distance_cm < TRIGGER_CM
    is_far = distance_cm > LEAVE_CM
    near_since = (near_since or now) if is_near else None
    far_since = (far_since or now) if is_far else None
    near_ok = near_since is not None and now - near_since >= HOLD_SEC
    far_ok = far_since is not None and now - far_since >= HOLD_SEC

    if state == "IDLE":
        text, color = "Step closer", GREY
        if near_ok:
            state, state_start = "CHECKING", now
            streak = helmet_frames = vest_frames = 0
            send("C")

    elif state == "CHECKING":
        text, color = "Checking PPE...", ORANGE
        helmet, vest, shown = detect(frame)
        streak = streak + 1 if (helmet and vest) else 0
        helmet_frames += int(helmet)
        vest_frames += int(vest)
        if streak >= FRAMES_NEEDED:
            send("G")
            log_event("PASS", [])
```

```
            state, state_start = "RESULT_PASS", now
        elif far_ok:
            send("I")
            state = "IDLE"
        elif now - state_start > CHECK_WINDOW_SEC:
            missing = []
            if helmet_frames < MIN_SEEN_FRAMES:
                missing.append("helmet")
            if vest_frames < MIN_SEEN_FRAMES:
                missing.append("vest")
            if not missing:
                # both were seen but not together long enough: give another window
                state_start = now
                streak = helmet_frames = vest_frames = 0
            else:
                send("R" if len(missing) == 2 else ("H" if "helmet" in missing else "V"))
                photo = save_snapshot(shown)
                log_event("VIOLATION", missing, photo)
                send_telegram_alert(missing, photo)
                state, state_start = "RESULT_FAIL", now

    elif state == "RESULT_PASS":
        text, color = "ACCESS GRANTED", GREEN
        if now - state_start > GRANT_OPEN_SEC:
            send("I")
            state = "WAIT_LEAVE"

    elif state == "RESULT_FAIL":
        text, color = "ACCESS DENIED", RED
        if now - state_start > DENY_SHOW_SEC:
            send("I")
            state = "WAIT_LEAVE"

    else:   # WAIT_LEAVE
        text, color = "Please step away", GREY
        if far_ok:
            state = "IDLE"

    cv2.rectangle(shown, (0, 0), (shown.shape[1], 40), color, -1)
    cv2.putText(shown, text + "   dist: " + str(distance_cm) + " cm", (10, 28),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
    cv2.imshow("PPE Compliance Gate (q to quit)", shown)

send("I")
cap.release()
cv2.destroyAllWindows()
if ser:
    ser.close()
```

How to run and test it

1. **Simulation first** (no hardware): set `SERIAL_PORT = None`, then run `python main.py`.
2. The terminal prints the **model's class names**. If helmet or vest classes are not covered by `HELMET_NAMES` and `VEST_NAMES`, add them (lower case).
3. Press **n**: the banner turns orange ("Checking PPE..."). Stand in front of the camera:
   - With helmet and vest: the banner turns green, and the terminal prints `-> Arduino: G`.
   - Without one or both: after about 3 seconds the banner turns red, a photo is saved in `snapshots/`, a line is added to `logs/events.csv`, and the alert arrives on Telegram.
4. Press **f** (person leaves), then **n** again to test the next person.
5. **With hardware:** set `SERIAL_PORT = "COM3"` (your port), close the Serial Monitor, and run again. Walking up to the sensor now starts the check.

Tuning

Problem	Change
Misses the helmet or vest	Lower `CONFIDENCE` to 0.35-0.4, or retrain with your own photos
Opens when it should not	Raise `CONFIDENCE`, or raise `FRAMES_NEEDED`
Laggy video	Set `IMG_SIZE = 480`, close other programs
Decision feels slow	Lower `FRAMES_NEEDED` to 3
Denied although PPE is worn	Raise `CHECK_WINDOW_SEC` to 4-5
Gate wakes too early or late	Change `TRIGGER_CM` (keep `LEAVE_CM` about 20 larger)
Gate wakes when nobody is there	Raise `HOLD_SEC`, and check the sensor is not aimed at a wall or table
Too many Telegram alerts	Raise `ALERT_COOLDOWN`

Phase G: The report dashboard (30 minutes)

Create `dashboard.py`:

```
import os

import pandas as pd
import streamlit as st

LOG_FILE = "logs/events.csv"

st.set_page_config(page_title="PPE Gate Report", layout="wide")
st.title("PPE Compliance Gate: Report")
if st.button("Refresh"):
    st.rerun()

if not os.path.exists(LOG_FILE):
    st.info("No events yet. Run main.py first, from the same folder.")
    st.stop()

df = pd.read_csv(LOG_FILE)
total = len(df)
passed = int((df["result"] == "PASS").sum())
violations = int((df["result"] == "VIOLATION").sum())
rate = 100 * passed / total if total else 0

c1, c2, c3, c4 = st.columns(4)
c1.metric("Total checks", total)
c2.metric("Compliant", passed)
c3.metric("Violations", violations)
c4.metric("Compliance rate", f"{rate:.0f}%")

viol = df[df["result"] == "VIOLATION"]
if not viol.empty:
    items = viol["missing"].fillna("").str.split("+").explode()
    st.subheader("What was missing most often")
    st.bar_chart(items[items != ""].value_counts())

st.subheader("Event log (latest first)")
st.dataframe(df.tail(50).iloc[::-1], use_container_width=True)

st.subheader("Latest violation photos")
photos = [p for p in viol["snapshot"].dropna().tolist()[-6:][::-1] if os.path.exists(p)]
cols = st.columns(3)
for i, p in enumerate(photos):
    cols[i % 3].image(p, caption=os.path.basename(p))
```

Run it in a second terminal with `streamlit run dashboard.py`. It opens in your browser. **Close** `events.csv` **if it is open in Excel**, or the main program cannot write to it.

Done when: the page shows the totals, the compliance rate, the chart and the photos.

Phase H: Test properly and collect numbers (2-3 hours)

Run each situation **20 times** and record how often the gate is right. Real numbers, including mistakes, impress examiners more than claims of 100%.

Situation	Correct result	Trials	Correct	Accuracy
Helmet + vest	Gate opens	20		
Helmet only	Denied, "vest" missing	20		
Vest only	Denied, "helmet" missing	20		
Neither	Denied	20		
Different people or lighting	Mixed	20		

Extra tests for the sensor and alerts:

Test	How	Record
Trigger distance	Stand at 50, 80, 100, 130 and 160 cm, 10 times each	How often the gate wakes at each distance
False triggers	Leave the gate idle for 5 minutes while people walk past	Number of unwanted wake-ups
Telegram delivery	Cause 10 violations, spaced beyond the cooldown	How many alerts arrived, and the average delay
LCD messages	Check all states	Whether the message matched the decision

Also keep for the report: the training graphs and mAP score (if you trained), screenshots of the dashboard and Telegram alerts, and a **backup video** of the gate working.

---

5. Prompts to give the Antigravity agent

Paste these into the agent panel, one at a time. Fill in the brackets. Always read the result before accepting it.

Task	Prompt
Set up	"Read AGENTS.md. Create requirements.txt for this project and tell me the commands to create a Python virtual environment on Windows and install it. Do not run anything yet."
Explain code	"Explain main.py to me line by line as if I am a beginner. Then list 10 questions an examiner might ask about it and give short answers."
Fix an error	"I ran python main.py and got this error: [paste the exact error]. Explain what it means in simple words, then fix it with the smallest possible change."
Wrong class names	"My model prints these class names: [paste]. Update HELMET_NAMES and VEST_NAMES in main.py so helmet and vest detection works. Do not change anything else."
Add a feature	"In main.py, add a counter on the video screen showing today's number of passes and violations. Keep the change small and tell me exactly which lines you changed."
Tune	"Detection of the vest is poor in my classroom. Suggest three settings in main.py I can change, explain each, and tell me how to test the effect."
Test script	"Write a small script that reads logs/events.csv and prints how many PASS and VIOLATION events there are, and the compliance rate."
Report help	"Write a 150-word explanation of the state machine in main.py (IDLE, CHECKING, RESULT_PASS, RESULT_FAIL, WAIT_LEAVE) for my project report."
Review	"Review main.py for bugs, unsafe handling of secrets, and anything that could crash during a live demo. List the issues in order of importance. Do not rewrite the file."

---

6. Troubleshooting

Problem	Fix
`pip install` fails or is very slow	Use Python 3.10 or 3.11 (not the newest), a stable connection, and retry
`python` is not recognised	Reinstall Python and tick "Add Python to PATH", then open a new terminal
The terminal does not show `(venv)`	Run `venv\Scripts\activate` again in that terminal
"could not open port COM3"	Close the Arduino Serial Monitor, check the COM number, replug the Arduino
No distance values	Check the final Arduino sketch is uploaded and the baud rate is 9600
Camera is black or will not open	Change `CAMERA_INDEX` to 1, and close Zoom, Teams or other apps using the camera
Never detects helmet or vest	Compare the printed class names with `HELMET_NAMES` and `VEST_NAMES`, then add your own photos (Phase C)
Model file error	Check `best.pt` is a real model (several MB) and in the same folder
Dashboard says no events	Run `main.py` first, from the same folder
`events.csv` cannot be written	Close it in Excel
Telegram `401`	The token is wrong. Copy it again from BotFather
Telegram "chat not found"	Press Start in the bot chat and fetch the chat ID again
No Telegram alert and no error	Check the internet, the cooldown, and that the photo was saved
College WiFi blocks Telegram	Use your phone's hotspot
Very slow video	Lower `IMG_SIZE`, close other programs, or use a faster laptop

---

7. What to be ready to explain in the viva

- **What YOLO is:** a fast object detector that finds objects in an image and draws boxes with a confidence score.
- **Why laptop plus Arduino:** the AI needs more computing power than an Arduino has. The Arduino drives the physical outputs and reads the sensor.
- **The serial protocol:** one letter per command (C, G, R, H, V, I) and `D:<cm>` for distance, at 9600 baud.
- **The state machine:** why the sensor starts the check, and why the gate waits for the person to leave.
- **Why the AI needs several frames in a row:** one frame can be a mistake; consecutive frames make the decision reliable.
- **Where the IoT is:** sensors, a decision, actuators, and a network alert through the Telegram Bot API.
- **If you reused the GitHub model:** say so, and say what is your own work: the hardware, the integration, the state machine, the sensor trigger, the alerts and your test results.
- **Limitations:** it checks visibility, not correct wearing; it works best for one person in steady lighting; Telegram needs internet; the ultrasonic sensor cannot tell a person from an object.

---

8. Suggested order of work

Order	What	Needs the hardware?
1	Install tools, set up Antigravity, `AGENTS.md`	No
2	Phases A and B (environment, camera test)	No
3	Phase C (get the model)	No
4	Phase E (Telegram bot)	No
5	Phase F in simulation mode (`SERIAL_PORT = None`)	No
6	Phase G (dashboard)	No
7	Phase D (Arduino tests)	**Yes**
8	Phase F with the real Arduino and tuning	**Yes**
9	Phase H (20-trial tests, screenshots, backup video)	**Yes**

If you are short of time: make the core gate work first (camera, model, Arduino outputs), then add the LCD messages, then Telegram, then the dashboard. A reliable demo beats a broken one with many features.